MARS Game Documentation

This report is an overview on the approach used to create a grid based game on `MARS`, a GUI, Java-based simulator for the MIPS assembly language (**vollmar2006mars**; **Sanderson˙Vollmar˙200** (learned partially thanks to **smart-uu**). Said game sits inside of a seven by seven grid, where, if the player `P` reaches a reward `R`, a scoreboard above the grid increments by five. The player object `P` can be moved with `w`, `a`, `s` and `d`. The player will keep moving in the last direction it was prompted to. If the scoreboard reaches one hundred points or the player touches the border, the game will terminate and output the final score.

Design and Implementation Overview

The program is organized in a few function callers and functions. These are portrayed in figure **??** somewhat unconventionally, with priority on the demonstration of the pathways taken within the code. Jumps in the diagram are depicted as arrows, with `jal` being colored red.

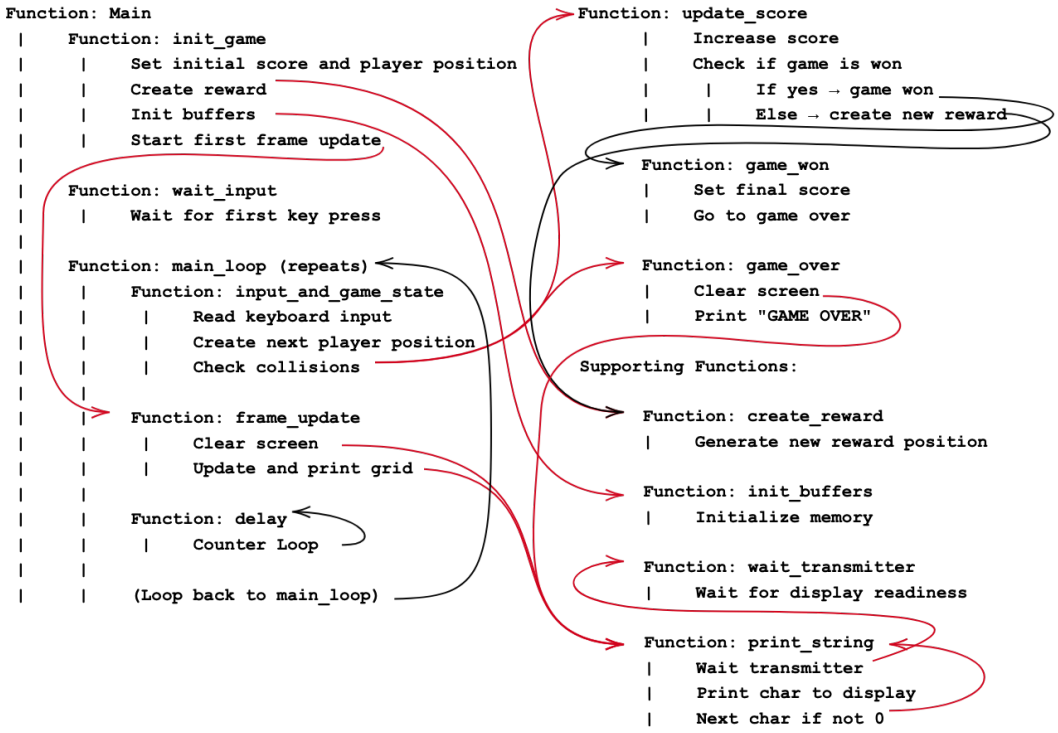

Figure 1: Code Diagram

Many of the design choices chosen have disadvantages, and some may render the program unusable given the device. In an itemized manner, the approach and the reason for it are explained for major code sections at a time.

`init_game`: Initializes the player environment and its variables by loading static memory into initialized memory, as well as prints said initial state. This is done by calling `create_reward`, `init_buffers` and `frame_update` before returning to `main`.

`wait_input`: Check for the display's ready bit and loops if still equal to 0. This is done so that the player doesn't accidentally start moving by itself.

`main_loop`: The primary game loop. As the main function caller, it calls upon `input_and_game_state`, `frame_update` and `delay` in that order, as to optimize response time from the last input.

`input_and_game_state`: Function caller that echoes the input, branches to a function to create the next player position depending on input and checks that next player position is valid (calling `create_reward` if applicable). Saves the updated grid.

`frame_update`: – Refreshes display by printing thirty new lines and reprints the grid. This does not erase the previous grid, but masks it for the use. Although not desirable, reduces program complexity. This was used instead of clearing the display due to the lack of suitable options found (even when printing all possible ASCII characters)(although one certainly existing).

`delay`: Loops the program a set amount of times (two hundred thousand times as default) in a simple decrement loop as to simulate a waiting function. This is the single bigger source of issue in this game since it is not even native (i.e. can be run-dependent depending on `MARS`'s initial stat). The counter is well highlighted in the code, and can be easily changed. Although undesired, this issue does mean that there is less code complexity. Reason for not using a clock for this action is for lack of suitable options found (although one certainly existing). Function does not exist in base game, disadvantages can be ignored.

`update_score`: This function takes the stored score value, adds 5 and manages any carry over to the decimal place. Includes a redundant function for "initiating" the decimal numbers. Avoids start string of having two zeros (`00`).

`game_over and game_won`: These manage termination.`game_over` prints a new screen (enters a newline repetitively), the current score and terminates, while `game_won` updates score to a hundred and then calls `game_over`.

Supporting functions as `create_reward`, `init_buffers`, `wait_transmitter`, and `print_string` provide the capability for essential low level operations such as, creating a valid `R`, managing memory and printing.

Design Justifications and Challenges

A primary design decision was to use a string for the grid and changing that directly in an initialized space. This was greatly beneficial due to the players position being easily replaced, and the string to be changed with the same stored value as the stored position values since they both are index based (created with help from **noauthor˙undated-eh**).

Most challenging design choice was the memory mapped input and output. Due to this, clearing the display was not coded cleanly and leads to a noticeable shift upwards on slow-running machines when updating the grid due to the repeated printing of newlines. The ladder is another cause of time inefficiency raised by the lack of display clearing.

The busy–wait delay loop, although imprecise and not consistent (even in the same system), still impresses in simplicity. Due to the program not waiting for the input and grabbing the last value in the keyboard register regardless, input response time is not affected.

Score conversion was another critical area. Early implementations of it resulted in misprints of the decimal number and wrong ASCII conversion. With help of ASCII tables (**wiki:xxx**) and robust logic, this issue has been overcome.

Conclusion

The design of this game is concise. A lot has been put into making the code more streamlined, and it reflects so when playing. Although certain aspects leave a lot on the table (such as the display not clearing and the time delay being a loop), its modular structure allows for extensions, modifications and a greater readability.

Further implementation detail can be found in the form of single line comments.